

Web制作勉強会 実践編

Drama ArchiveをNext.jsで再設計する

期間：2026年9月11日（金）から約2か月

題材：受講者が制作した「Drama Archive」

対象：HTML・CSS・JavaScriptの基礎学習を終え、今後Web部門の制作を担う制作者

進行：講師が完成形を一方向的に教えるのではなく、受講者と伴走者が一つの制作物を一緒に改善する

はじめに

この勉強会では、すでに完成している静的サイト「Drama Archive」をNext.jsへ移植します。

目的は、Next.jsの機能を暗記することではありません。また、元のコードを古いものとして否定することでもありません。

受講者が自分で考え、デザインし、HTML・CSS・JavaScriptで完成させたページを出発点にして、次のことを体験します。

- 一つの長いHTMLを、役割ごとの部品へ分ける
- 同じ形の要素を、データから繰り返し生成する
- DOMを直接操作するJavaScriptを、Reactの状態管理へ置き換える
- CSSを、複数人で維持しやすいSCSS構成へ整理する
- モーダルやスクロールアニメーションを、他の案件でも使える仕組みにする
- Gitで変更理由と制作履歴を残す
- 完成したサイトを公開し、次の案件へ知識を引き継ぐ

最終的な成果物は、単に「Next.jsで動くDrama Archive」ではありません。

この制作を、会社のWeb部門が今後使える制作基準の最初の実例にします。

この教材の使い方

本編は全14回を想定しています。1回で完全に理解することよりも、実際に動かし、壊し、直し、理由を言葉にできることを重視します。

各回は、原則として次の順に進めます。

```
前回からの制作状況を共有する
↓
今回扱う考え方を確認する
↓
実際のDrama Archiveへ適用する
↓
ブラウザとコードを確認する
↓
Gitへ記録する
```



判断理由を短くREADMEへ残す

分からないことが出た場合は、AIも制作道具として利用します。ただし、返ってきたコードをそのまま採用するのではなく、少なくとも次を確認します。

- 何を解決するコードなのか
- どのファイルに置くのか
- 既存のコードへどんな影響があるのか
- エラー時に元へ戻せるか
- 自分たちの制作基準に合うか

事前準備編について

環境整備編は、すでに作成した「Web制作勉強会00」を使用します。

そこで準備したものを本編でも継続して使います。

- VS Code
- Node.js / npm
- Git
- GitHub
- SSH接続
- [drama_archive](#)リポジトリ
- Next.jsプロジェクト
- ESLint、Prettier、Stylelintなどの制作補助

本教材では環境構築を繰り返しません。開始時点で、`npm run dev`によりNext.jsの初期ページを表示できるものとします。

題材となるコードの評価

Drama Archiveには、すでに次の能力が表れています。

- HTMLの大きな構造を組み立てられる
- CSS GridとFlexboxを使い分けられる
- モバイルファーストでレスポンス設計ができる
- CSS変数、`clamp()`、`aspect-ratio`、`object-fit`を使える
- JavaScriptで複数要素へイベントを設定できる
- `data-*`属性とIDを対応させられる
- モーダルとモバイルメニューを実装できる
- デザイン、画像、文章を一つの作品としてまとめられる

したがって、本編をHTML・CSSの初歩へ戻す必要はありません。

一方、組織制作へ進むための次の課題も見えます。

- 作品ごとにカードとモーダルのHTMLを書かないといけない

- データに変更があるとHTMLへも反映させなければいけない
- クリックやタップだけでなく、キーボードでも操作できるようにする
- 背景スクロール停止がない
- PCとモバイルの見た目を分けつつ、リンク情報を共通化したほうがいい

本編では、この差を埋めます。

完成時の構成（案）

```
/
├── public/
│   └── pict/
│       ├── logo.svg
│       └── 各作品の画像.avif
├── src/
│   ├── app/
│   │   ├── favicon.ico
│   │   ├── globals.scss
│   │   ├── layout.js
│   │   ├── page.js
│   │   └── page.module.scss
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Header/
│   │   │   │   ├── index.jsx
│   │   │   │   └── Header.module.scss
│   │   │   ├── Footer/
│   │   │   │   ├── index.jsx
│   │   │   │   └── Footer.module.scss
│   │   │   └── MobileMenu/
│   │   │       ├── index.jsx
│   │   │       └── MobileMenu.module.scss
│   │   ├── contents/
│   │   │   ├── DramaArchive.jsx
│   │   │   ├── DramaArchive.module.scss
│   │   │   ├── FeaturedDrama.jsx
│   │   │   ├── FeaturedDrama.module.scss
│   │   │   ├── DramaList.jsx
│   │   │   ├── DramaList.module.scss
│   │   │   ├── DramaCard.jsx
│   │   │   ├── DramaCard.module.scss
│   │   │   ├── DramaDetail.jsx
│   │   │   └── DramaDetail.module.scss
│   │   └── ui/
│   │       ├── Modal/
│   │       │   ├── index.jsx
│   │       │   └── Modal.module.scss
│   │       └── ScrollReveal/
│   │           └── index.jsx
```

```
├── ScrollReveal.module.scss
├── data/
│   └── dramas.js
├── styles/
│   ├── _index.scss
│   ├── _base.scss
│   ├── reset.css
│   └── shared/
│       ├── _index.scss
│       ├── _variables.scss
│       └── _mixins.scss
├── .gitignore
├── eslint.config.mjs
├── jsconfig.json
├── next.config.mjs
├── package.json
├── package-lock.json
└── README.md
```

この構成は唯一の正解ではありません。重要なのは、置き場所を見ただけで「ページ」「データ」「共通UI」「スタイル」のどれかを判断できることです。

第1章 完成サイトを設計図として読む

目的

Next.jsへ移す前に、元サイトを壊さず保存し、ページの構成と動きを説明できる状態にします。

1. 元サイトを保存する

完成済みの静的サイトは、`tmp/drama_archive` へ置き、移植中の比較対象として残します。直接上書きしません。

元サイト = 正解の見た目と動き
Next.js版 = これから育てる実装

2. ページを部品として見る

元のHTMLを、次の単位に分けて考えます。

```
Header
├── Logo
├── MenuButton
└── Navigation
```

Main

```
├── Featured
├── DramaList
│   ├── DramaCard
│   └── DramaModal
└── About
```

Footer

今日の確認

- ☐ 元サイトを直接編集しない状態にした

Git

```
git add README.md
git commit -m "開発環境を整える"
```